Avance de Proyecto Final 1

Sistema de Matrícula

Curso: Herramientas de Desarrollo (100000S66T)
Periodo: 2026-II
Semana: 6
Proyecto: Sistema de Matrícula
Repositorio: github.com/Rodrigo2326/sistema-matricula

Integrantes:
Ccahuana Huillca Giancarlo
Aparcana Mamani Rodrigo Alonso
Arnold Jhuncor Díaz Silva

1. Presentación del proyecto

El proyecto consiste en una aplicación web para el registro y consulta de matrículas de estudiantes por ciclo académico. Se ejecuta directamente en el navegador, sin servidor ni instalación previa, y guarda la información en el almacenamiento local del navegador. Se eligió deliberadamente una solución pequeña y autocontenida para que el esfuerzo del curso se concentre en la gestión técnica del desarrollo —versionado, ramas, integración y trazabilidad— y no en la complejidad de la programación.

Este primer avance corresponde al objetivo del APF1: establecer una base de trabajo versionada y demostrar el flujo fundamental de Git, desde el repositorio local hasta la colaboración básica con un repositorio remoto.

2. Problema o necesidad

Las áreas académicas pequeñas siguen registrando las matrículas de cada ciclo en hojas de cálculo compartidas: el archivo se duplica entre varias personas, no existe validación de los datos ingresados, un mismo estudiante termina registrado dos veces y consultar quién está matriculado en un curso obliga a revisar el archivo fila por fila. Este proyecto reemplaza esa hoja de cálculo por una aplicación web sencilla que centraliza el registro, valida la información al momento de ingresarla y permite buscar y exportar los datos sin depender de quién tenga la última versión del archivo.

3. Objetivo del proyecto

Construir una solución web funcional para el registro y consulta de matrículas, gestionada íntegramente con las herramientas de control de versiones y colaboración trabajadas en el curso, de modo que la evolución del producto quede trazable en el repositorio a lo largo de todo el semestre.

4. Alcance de esta primera versión

4.1. Incluido en el APF1

- Registro de matrículas mediante formulario con siete campos.
- Listado de las matrículas registradas en tabla, con contador de registros.
- Búsqueda y filtrado del listado por nombre, código, carrera o curso.
- Validación de los datos antes de guardar el registro.
- Exportación del listado a archivo CSV.
- Resumen de matrículas agrupado por carrera.
- Eliminación de registros con confirmación previa.
- Persistencia de la información en el navegador.

4.2. Fuera de alcance en esta versión

Autenticación de usuarios, base de datos en servidor, integración continua, contenedores y despliegue en nube. Estos puntos corresponden a los avances APF2, APF3 y al proyecto final, según la ruta de entregas del curso.

5. Estructura de la aplicación

La aplicación separa la interfaz, los estilos y la lógica en archivos independientes, de modo que cada cambio afecte solo al archivo que le corresponde y los conflictos de integración queden acotados:

```
sistema-matricula/
■■■ index.html                  Interfaz de la aplicación
■■■ css/
■   ■■■ estilos.css             Hoja de estilos
■■■ js/
■   ■■■ app.js                  Lógica de la aplicación
■■■ docs/
■   ■■■ informe-apf1.md         Este informe
■   ■■■ Informe_APF1.pdf        Este informe en PDF
■   ■■■ generar_informe.py      Script que genera el PDF
■   ■■■ evidencias/             Salidas de Git que respaldan el avance
■■■ .gitignore
■■■ README.md
```

El archivo `js/app.js` está organizado en bloques comentados que corresponden uno a uno con las ramas de trabajo: almacenamiento y avisos, registro, listado, búsqueda, validación, exportación a CSV y resumen por carrera.

6. Funcionalidades operativas

#	Funcionalidad	Descripción	Rama que la implementó
1	Registro de matrícula	Formulario con nombres, apellidos, código, DNI, carrera, ciclo y curso. Guarda el registro y confirma con un aviso.	`feature/formulario-matricula`
2	Listado de matrículas	Tabla con los registros, contador y eliminación con confirmación.	`feature/listado-matriculas`
3	Búsqueda	Filtra la tabla mientras se escribe, por nombre, código, carrera o curso.	`feature/busqueda-matriculas`
4	Validación	Verifica formato de código y DNI, campos obligatorios y matrículas duplicadas.	`feature/validacion-formulario`
5	Exportación a CSV	Descarga el listado como archivo CSV compatible con Excel.	`feature/exportar-csv`
6	Resumen por carrera	Panel desplegable con el total de matriculados en cada carrera.	`feature/resumen-por-carrera`
7	Edición de matrícula	Permite corregir un registro ya guardado sin eliminarlo y volver a crearlo.	`feature/editar-matricula`
8	Filtros por carrera y ciclo	Dos selectores que se combinan entre sí y con la búsqueda de texto.	`feature-filtros-ordenamiento`
9	Ordenamiento del listado	Ordena por cualquier columna al hacer clic en su encabezado.	`feature-filtros-ordenamiento`

El avance supera holgadamente el mínimo exigido de tres funcionalidades identificables. Las funcionalidades 1 a 6 corresponden a Rodrigo Aparcana, la 7 a Giancarlo Ccahuana y las 8 y 9 a Arnold.

6.1. Reglas de validación aplicadas

- Nombres y apellidos no pueden quedar vacíos.
- El código de estudiante debe seguir el formato `2026-0001` (cuatro dígitos, guion, cuatro dígitos).

- El DNI debe tener exactamente ocho dígitos.
- El curso a matricular es obligatorio.
- Un mismo estudiante no puede matricularse dos veces en el mismo curso.

Cuando un dato no cumple una regla, la aplicación muestra el motivo concreto y no guarda el registro.

7. Gestión del repositorio

7.1. Inicialización y repositorio remoto

El repositorio se inicializó localmente con `git init -b main` y se publicó en GitHub como repositorio público. La rama principal es `main` desde el primer commit, sin renombrados posteriores.

```
git init -b main
git add .
git commit -m "Agregar estructura base del proyecto"
gh repo create sistema-matricula --public --source=. --remote=origin --push
```

Repositorio remoto: **https://github.com/Rodrigo2326/sistema-matricula**

7.2. Convenciones adoptadas

Ramas. Se siguen las convenciones mínimas recomendadas en los lineamientos:

- `main` — versión principal estable.
- `feature/<nombre-funcionalidad>` — nuevas funcionalidades.
- `fix/<nombre-error>` — correcciones.
- `hotfix/<nombre-problema>` — correcciones urgentes, si corresponde.

Mensajes de commit. Cada mensaje empieza con un verbo en infinitivo y describe el cambio concreto ("Agregar formulario de registro de matrícula", "Corregir pérdida del filtro al registrar o eliminar una matrícula"). Se evitaron deliberadamente mensajes genéricos como "cambio", "prueba" o "commit 1". Los commits que lo ameritan incluyen un cuerpo que explica el motivo del cambio, no solo el qué.

Integración. Todas las ramas se integran a `main` con `git merge --no-ff`, de modo que el historial conserve el punto de fusión y se pueda identificar qué commits pertenecieron a cada funcionalidad.

8. Ramas de trabajo

Se crearon dieciocho ramas de trabajo, cada una asociada a una funcionalidad o corrección concreta. Ninguna rama se creó sin cambios reales asociados. La tabla siguiente recoge las principales; el listado completo, con el responsable de cada una, está en `docs/evidencias/ramas.txt`.

Rama	Propósito
`feature/formulario-matricula`	Formulario de registro
`feature/listado-matriculas`	Tabla de registros y eliminación
`feature/busqueda-matriculas`	Filtrado del listado
`feature/validacion-formulario`	Reglas de validación
`feature/exportar-csv`	Exportación a CSV
`feature/resumen-por-carrera`	Resumen agrupado por carrera
`fix/filtro-se-pierde-al-actualizar`	Corrección de un error detectado
`feature/documentacion-apf1`	Informe y evidencias del avance

Rama	Propósito
`feature/editar-matricula`	Edición de matrículas (Giancarlo Ccahuana)
`feature-filtros-ordenamiento`	Filtros y ordenamiento (Arnold)

La evidencia completa está en `docs/evidencias/ramas.txt`, que indica además el responsable de cada rama.

9. Historial de commits y gráfico de ramas

El repositorio acumula commits de trabajo más los commits de fusión generados por cada integración. El historial completo, con autor y fecha, está en `docs/evidencias/historial-commits.txt`, y el gráfico de ramas en `docs/evidencias/grafico-ramas.txt`.

El gráfico muestra el patrón esperado: desde `main` se abre cada rama de trabajo, se realizan uno o más commits en ella y se vuelve a `main` mediante un commit de fusión. Las ramas `feature/exportar-csv` y `feature/resumen-por-carrera` aparecen partiendo del mismo punto, que es el origen del conflicto descrito en la sección 11.

10. Evidencia de las fusiones

Todas las integraciones a `main` se realizaron con `git merge --no-ff`. El detalle está en `docs/evidencias/merges.txt`.

Ejemplo de una fusión sin conflicto:

```
$ git checkout main
$ git merge --no-ff feature/busqueda-matriculas
Merge made by the 'ort' strategy.
 css/estilos.css | 7 +++++++
 index.html      | 1 +
 js/app.js       | 24 ++++++++++++++++++++++
```

Se utiliza `--no-ff` de forma deliberada: si Git resolviera la integración con un avance rápido (*fast-forward*), los commits de la rama quedarían mezclados en la línea principal y se perdería la información de qué trabajo perteneció a qué funcionalidad.

11. Evidencia del conflicto y su resolución

11.1. Cómo se generó

El conflicto se produjo de forma controlada, reproduciendo una situación habitual de trabajo en equipo: dos personas desarrollan funcionalidades distintas al mismo tiempo, partiendo de la misma versión de `main`, y ambas modifican la misma zona del archivo.

Las ramas `feature/exportar-csv` y `feature/resumen-por-carrera` se crearon desde el mismo commit (`2f174b0`). Ambas agregaron un botón nuevo en la misma línea de la barra de herramientas del listado y anexaron código al final de `css/estilos.css` y `js/app.js`.

La primera rama se integró sin problemas. Al integrar la segunda, Git no pudo decidir cuál de los dos cambios conservar:

```
$ git merge --no-ff feature/resumen-por-carrera
Auto-merging css/estilos.css
CONFLICT (content): Merge conflict in css/estilos.css
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Auto-merging js/app.js
CONFLICT (content): Merge conflict in js/app.js
Automatic merge failed; fix conflicts and then commit the result.

$ git status --short
UU css/estilos.css
UU index.html
UU js/app.js
```

Git dejó las regiones afectadas marcadas en el área de trabajo:

```
<<<<<<< HEAD
  <button type="button" class="btn-secundario" id="btnExportar">Exportar CSV</button>
=======
  <button type="button" class="btn-secundario" id="btnResumen">Ver resumen</button>
>>>>>>> feature/resumen-por-carrera
```

11.2. Cómo se resolvió

El criterio fue analizar si los cambios se excluían entre sí. No era el caso: exportar a CSV y ver el resumen por carrera son funcionalidades complementarias, y descartar cualquiera de las dos habría significado perder trabajo ya realizado. Por eso la resolución conservó el contenido de ambas ramas:

- `index.html` — se mantuvieron los dos botones dentro de la misma barra de herramientas.
- `css/estilos.css` — se conservaron los estilos del panel de resumen. La regla `.btn-secundario` era idéntica en ambas ramas, por lo que Git la unificó automáticamente sin intervención.
- `js/app.js` — se conservaron los dos bloques de lógica.

Como la barra pasó a tener dos botones, se ajustó además la regla de alineación para que solo el primero empuje el grupo hacia la derecha. Este detalle ilustra algo que conviene señalar: resolver un conflicto no siempre es elegir entre dos versiones, a veces exige un tercer cambio que haga que ambas convivan correctamente.

Una vez eliminados los marcadores, la fusión se confirmó:

```
$ git add index.html css/estilos.css js/app.js
$ git commit
```

11.3. Verificación posterior

- No quedaron marcadores de conflicto en el árbol de trabajo.
- La barra muestra los dos botones y ambos responden correctamente.
- El listado, la búsqueda y la validación siguieron operativos.

Las evidencias están en `docs/evidencias/conflicto-01-deteccion.txt`, `conflicto-02-marcadores.txt` y `conflicto-03-resolucion.txt`.

12. Corrección derivada del avance

Durante la integración se detectó un error real: al registrar o eliminar una matrícula con el buscador activo, la tabla se redibujaba con la lista completa. El texto seguía visible en el buscador, pero el listado mostraba todos los registros, lo que resultaba confuso.

La causa era que ambas operaciones llamaban a `renderizarTabla(matriculas)` con la lista completa en lugar de aplicar el filtro vigente. Se corrigió en la rama `fix/filtro-se-pierde-al-actualizar`, agregando la función `refrescarListado()` que vuelve a dibujar la tabla respetando lo escrito en el buscador.

Este caso es relevante para el avance porque muestra el ciclo completo: detectar un error, aislarlo en una rama `fix/`, corregirlo con un commit descriptivo e integrarlo a `main`.

13. Sincronización entre repositorio local y remoto

El flujo aplicado en cada funcionalidad fue el siguiente:

```
git checkout main                    # ubicarse en la rama principal
git pull                             # traer los cambios del remoto
git checkout -b feature/nueva        # crear la rama de trabajo
# ... edición de archivos ...
git add .                            # preparar los cambios
git commit -m "Agregar nueva funcionalidad"
git push -u origin feature/nueva     # publicar la rama en el remoto
git checkout main                    # volver a la principal
git merge --no-ff feature/nueva      # integrar conservando el punto de fusión
git push origin main                 # sincronizar la integración
```

Todas las ramas de trabajo están publicadas en el repositorio remoto, no solo en local, de modo que el historial completo es verificable desde GitHub. El estado de sincronización está registrado en `docs/evidencias/sincronizacion-local-remoto.txt`.

14. Participación del equipo

Integrante	Usuario de GitHub	Rol	Commits
Aparcana Mamani Rodrigo Alonso	@Rodrigo2326	Responsable de repositorio y documentación	39
Ccahuana Huillca Giancarlo	@soulahrikermette-hub	Responsable funcional	6
Arnold Jhuncor Díaz Silva	@gamergggpro123-lgtm	Responsable de calidad y automatización	5

Los tres integrantes están registrados como colaboradores del repositorio y tienen commits propios en el historial, verificables con `git shortlog -sn` y en la pestaña *Contributors* de GitHub.

El repositorio incluye un archivo `.mailmap` porque dos integrantes configuraron Git con más de un nombre sobre el mismo correo, y el historial los contaba como personas distintas. El mapeo corrige el recuento en `git log`, `git shortlog` y `git blame` sin reescribir ningún commit ni alterar sus identificadores.

14.1. Reparto del trabajo

Rodrigo Aparcana levantó la estructura del proyecto y las funcionalidades iniciales: formulario de registro, listado, búsqueda, validación, exportación a CSV y resumen por carrera. También se encargó de la configuración del repositorio, las convenciones de ramas y commits, y la documentación del avance.

Giancarlo Ccahuana desarrolló la edición de matrículas en la rama `feature/editar-matricula`. Era la carencia funcional más evidente de la primera versión: se podía crear y eliminar un registro, pero no corregirlo, de modo que un error de tipeo obligaba a borrar y volver a registrar. Su implementación resolvió además un detalle sutil: la validación de duplicados debía excluir el propio registro en edición, porque de lo contrario el sistema rechazaba guardar un registro por considerarlo duplicado de sí mismo.

Giancarlo se encargó además del ajuste visual de la barra de herramientas y del listado en la rama `feature/arreglar-toolbar`, que había quedado desalineada al sumar los selectores de filtro.

Arnold desarrolló los filtros por carrera y ciclo y el ordenamiento del listado en la rama `feature-filtros-ordenamiento`. El punto técnico más interesante de su aporte fue el ordenamiento por fecha: las fechas se almacenan como texto en formato `DD/MM/YYYY`, por lo que compararlas directamente producía un orden alfabético incorrecto. Su solución convierte cada fecha a milisegundos antes de compararlas, de modo que el orden resultante es cronológico.

14.2. Integración y revisión

Cada rama se revisó antes de integrarla a `main`, ejecutando la aplicación en un navegador y comprobando tanto la funcionalidad nueva como las anteriores. Ese control detectó cuatro defectos, que fueron corregidos antes de la fusión:

- Un identificador ausente en el HTML que impedía que el botón de envío cambiara su texto al entrar en modo edición.
- Una función `refrescarListado` declarada dos veces, cuya segunda declaración anulaba el ordenamiento por efecto del *hoisting* de JavaScript.
- Un elemento `id="contador"` duplicado en la barra de herramientas.
- Un campo de búsqueda reducido a unos pocos píxeles. El origen no estaba en las reglas de la barra sino en la regla general `input, select { width: 100% }` de la hoja de estilos, que hacía que los campos reclamaran el ancho completo del contenedor.

Las dos ramas se integraron sin conflictos entre sí, y las pruebas de las funcionalidades originales volvieron a ejecutarse después de la integración sin detectar regresiones.

15. Conclusiones

1. El proyecto cuenta con una base versionada correcta: repositorio inicializado con `main` como rama principal, remoto público configurado y dieciocho ramas de trabajo asociadas a funcionalidades concretas.
2. La separación de la aplicación en `index.html`, `css/estilos.css` y `js/app.js` no fue solo una decisión de orden: redujo el alcance de los conflictos, que quedaron acotados a las zonas realmente modificadas.
3. El conflicto de integración confirmó la utilidad de `--no-ff`. Al conservar los puntos de fusión, fue posible identificar con precisión qué dos ramas entraron en conflicto y desde qué commit partieron.
4. El trabajo en paralelo de tres personas sobre el mismo archivo confirmó el valor de revisar antes de integrar. Los cuatro defectos detectados no eran visibles leyendo el código: aparecieron al ejecutar la aplicación. Ninguno habría sido evidente en una revisión superficial del historial.

16. Próximos pasos (APF2)

- Sustituir la integración directa por Pull Requests con revisión previa, de modo que las observaciones queden registradas en el repositorio y no fuera de él.
- Publicar una Release que identifique la versión del proyecto.
- Incorporar un tablero de actividades con responsables y estados.
- Agregar al menos una funcionalidad nueva o una mejora importante.